

Digital Logic Circuits 1

HW IV



학번

2022202109

이름

이호정

Problem Statement

40 비트 Ripple Carry Adder 를 Verilog 로 구현하는 것이 문제이다. 먼저 Ripple Carry Adder 란 두 개의 이진수를 더하는 회로이다. 줄여서 RCA 라고 부르기도 한다. RCA 는 여러 개의 full-adder(전가산기) 를 직렬로 연결하여 연산을 수행한다. 물론 하고자하면 반가산기 (half-adder)를 끼워서 회로를 만들 수도 있긴 하다. 여기서 전가산기와 반가산기의 차이를 설명하자면 half-adder (반가산기) 는 두 개의 단일 비트 이진수를 더하는 입력회로이다. full-adder (전가산기) 는 두 비트의 합 과 이전 자리의 자리 올림인 C_{in} 을 고려하여 더해준다는 차이점이 있다. 따로 주의 할 점은 0 번째부터 시작한다는 것이다.

Descriptions for inputs and outputs

Input

A : 40 bit 크기의 이진수

B : 40 bit 크기의 이진수

C_{in} : 1bit 자리올림 입력

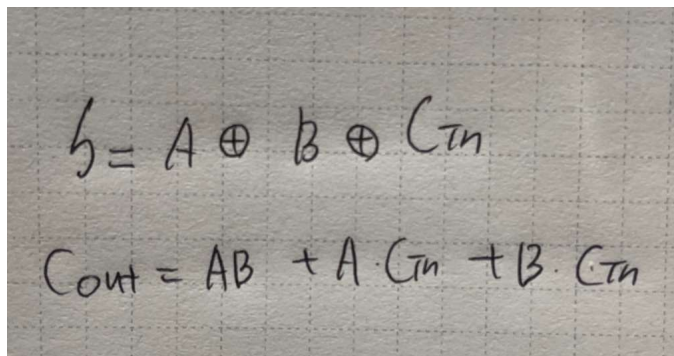
Output

S : 40bit 의 각 자리 합

C_{out} : 1bit 자리올림 출력
(carry)

Gate-level design and schematic of a full adder

Full-adder 의 출력은 다음과 같이 나오는데 과정은 아래와 같다.


$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + A \cdot C_{in} + B \cdot C_{in}$$

Input			Output	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full-adder 의 진리표

S

A \ $B C_{in}$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$S = A \oplus B \oplus C_{in}$

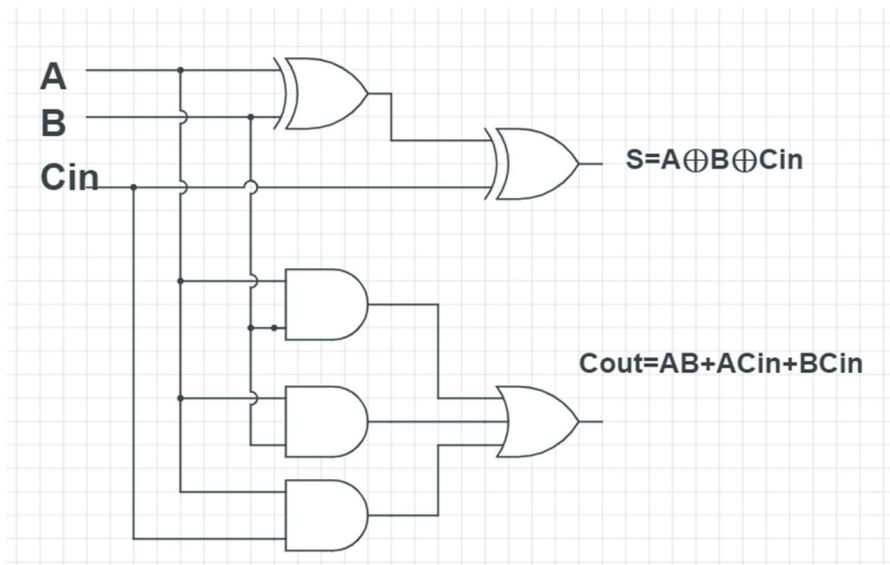
Sum 에 대한 k-map

C_{out}

A \ $B C_{in}$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$C_{out} = AB + AC_{in} + BC_{in}$

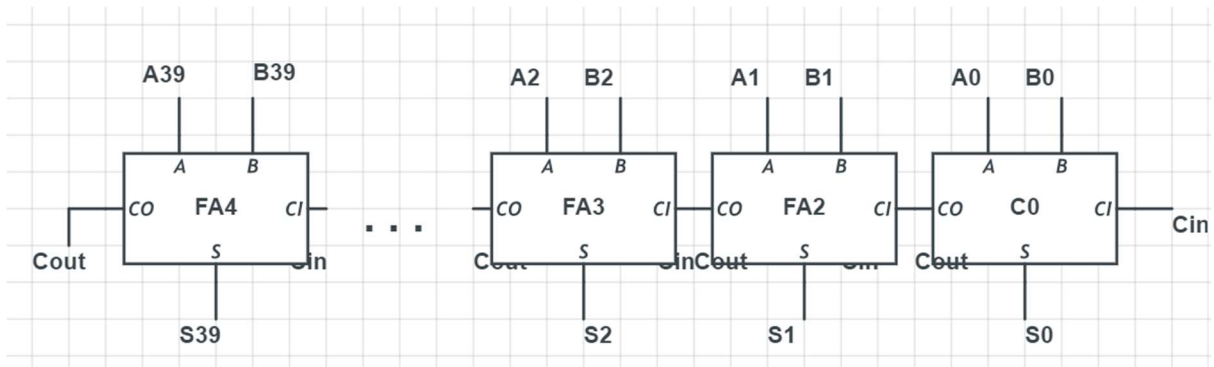
C_{out} 에 대한 k-map



Full-adder circuit

Block diagram of RCA

전가산기의 블록 다이어그램을 만들면 다음과 같다.



각 전가산기의 A 와 B 는 더하고자하는 두 수 의 각 비트이다. A0, B0 는 가장 작은 비트이며 숫자가 커질수록 큰 비트이고 이 회로에서는 A39, B39 이 가장 큰 비트이다. LSB , MSB

각 자리의 입력 A,B 를 더하고 이전 자리의 Carry 를 더한 결과가 Sum, S 로 출력된다. 그리고 덧셈 과정에서 캐리가 발생했다면 Cout 으로 출력된다.

Why do you need Cin?

Cin 이 필요한 이유는 , 여러 비트의 계산을 수행해야 하기 때문이다. 여러 비트의 이진수를 계산하면 자리올림이 발생하는 경우가 있는데, 이것을 고려하지 않고 각 자리의 비트에서 덧셈만 수행한다면 최종 덧셈이 올바르게 되지 않는다. 이전 비트에서 자리올림 carry 가 발생 했는지 안했는지 꼭 확인하고 이를 비트연산에 포함 시켜 주어야 한다.

What is the value of Cin?

첫 번째 전가산기의 Cin 은 외부에서 입력 받는다. 이전 비트가 없는 최초의 비트이기 때문이다. 보통 일반적으로 0 을 입력받지만 아닌 경우도 존재한다. 그리고 첫 번째 가산기 이후 부터는 이전 비트의 자리 올림 출력값인 Cout 을 Cin 으로 입력받는다.

Example

예를 들어 보자 아래와 같은 입력을 회로에 넣었다고 했을 때

입력

A: 1000000000 0000000000 0000000000 0000000011

B: 1000000000 0000000000 0000000000 0000001001

Cin: 0

덧셈 결과

0000000000 0000000000 0000000000 0000001100

39 번째 비트에서 자리올림이 발생하였으므로 Cout = 1

합(S)

0000000000 0000000000 0000000000 0000001100

자리올림(Cout): 1

Code analysis

```
module rca40 (
    output [39:0] S, // 40비트 덧셈 결과 출력
    input [39:0] A, B, // 두 개의 40비트 입력
    input Cin, // 초기 자리올림 입력
    output Cout // 최종 자리올림 출력
);
```

rca40 모듈 선언

40 비트 덧셈기. 출력은 40 비트 합(S)과 최종 자리올림(Cout), 입력은 두 개의 40 비트 숫자(A, B)와 초기 자리올림(Cin).

```
...wire [38:0] c; // 중간 자리올림을 저장할 39비트 와이어
```

c 라는 39 비트 와이어 배열을 선언하여 중간 자리올림 값을 저장 한다.

```
...// 첫 번째 비트에 대해 full_adder 사용
...full_adder fa0 (
    .....a(A[0]),
    .....b(B[0]),
    .....cin(Cin),
    .....cout(c[0]),
    .....sum(S[0])
    .....);
```

첫 번째 Full Adder 인스턴스: A[0], B[0], 그리고 Cin 을 입력으로 받아, S[0]에 합을 출력하고 c[0]에 자리올림을 출력한다.

```
...//중간 비트들에 대해 full_adder 사용
...generate
    .....genvar i;
    .....for (i=1; i<39; i=i+1) begin: full_adder_gen
    .....full_adder fa (
    .....a(A[i]),
    .....b(B[i]),
    .....cin(c[i-1]),
    .....cout(c[i]),
    .....sum(S[i])
    .....);
    .....end
...endgenerate
```

중간 비트들을 처리하기 위한 generate 블록 이다.

genvar i 선언: 반복문에서 사용할 변수 i를 만들어 선언하고

for 루프: i를 1 부터 38 까지 증가시키며, 각 비트에 대해 Full Adder 를 인스턴스화.

full_adder 인스턴스: 각 비트 A[i], B[i]와 이전 자리의 자리올림 c[i-1]을 입력으로 받아, S[i]에 합을 출력하고 c[i]에 자리올림을 출력 한다.

```

...//마지막 비트에 대해 full_adder 사용
...full_adder fa39 (
...    .a(A[39]),
...    .b(B[39]),
...    .cin(c[38]),
...    .cout(Cout),
...    .sum(S[39])
...);

```

마지막 Full Adder 인스턴스: A[39], B[39], 그리고 c[38]을 입력으로 받아, S[39]에 합을 출력하고 최종 자리올림을 Cout에 출력한다.

```
endmodule
```

rca40 모듈 종료하는 코드

```
module full_adder(a,b,cin,cout,sum);
```

full_adder 모듈 선언: 1 비트 Full Adder. 입력은 a, b, cin, 출력은 cout와 sum이다

```

...input a, b;
...input cin;
...output cout, sum;

```

input과 output 선언: a, b, cin은 입력 신호, cout와 sum은 출력 신호이다.

```

...assign cout = (a & b) | (a & cin) | (b & cin);
...assign sum = a ^ b ^ cin;

```

cout 계산

assign cout = (a & b) | (a & cin) | (b & cin);

자리올림은 a와 b가 모두 1이거나, a와 cin이 모두 1이거나, b와 cin이 모두 1일 때 발생한다.

sum 계산:

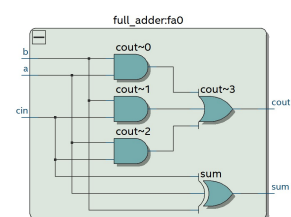
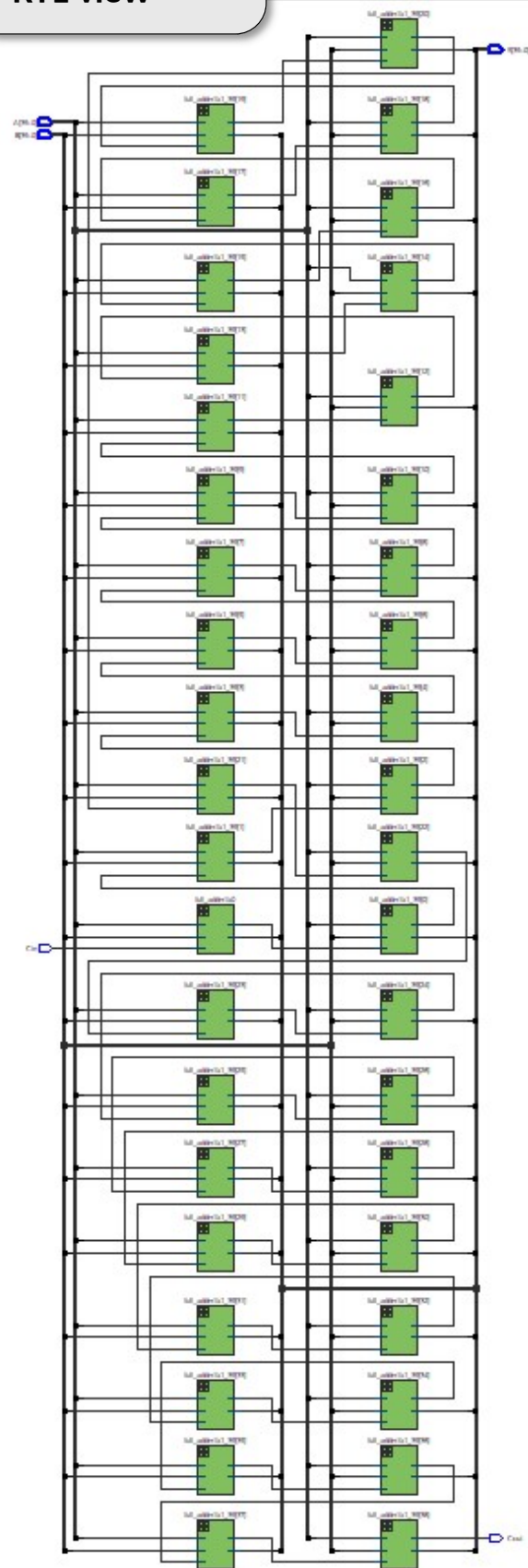
assign sum = a ^ b ^ cin;

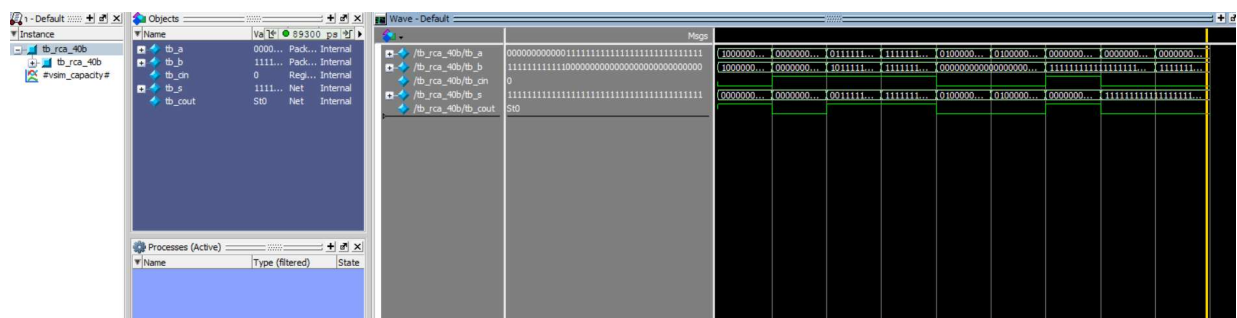
합은 a, b, cin의 XOR 결과임

```
endmodule
```

full_adder 모듈 종료

RTL view





Verification strategy & corresponding examples

Testbench 1

기본적인 덧셈 기능 확인

Input

A = 01000000000000000111001111000000000000

B = 000000000000000000000000000000001110111111101

$$C_{in} = 1$$

Output

S = 010000000000000000011100111101110111111110

Cout = 0

[illegible]

정상적으로 기본 덧셈이 되는 것을 확인 할 수 있다. 이것을 첫 번째 테스트 벤치로 정한 이유는 CA 가 두 입력과 Cin 값을 올바르게 더하여 예상되는 출력을 생성하는지 확인하기 위해서 이다.

Testbench 2

40 번째 full adder 에서 carry 가 발생했을 때 C_out 으로 정상인 값이 출력되는지 확인

Input

[illegible][illegible]
$$C_{in} = 0$$

Output

S = 00000000000000000000000000000000000001110

Cout = 1

Registers window showing the register list on the left and the selected register's value in the main pane. The register list includes:

- + /tb_rca_40b/tb_a
- + /tb_rca_40b/tb_b
- /tb_rca_40b/tb_cin
- + /tb_rca_40b/tb_s
- /tb_rca_40b/tb_cout

The main pane displays the value of the selected register, `tb_rca_40b/tb_cout`, as `St0` in hexadecimal.

40 번째에서 carry 가 발생했을 때, C_out 의 값이 1 로 정상적으로 출력되는 것을 확인할 수 있다.

이것을 테스트 벤치로 정한 이유는 40 번째 비트에서 캐리가 발생한 극단적인 경우를 테스트 해서 코드를 검증하기 위해서 이다.

Testbench 3

Cin 의 반영 확인_ Cin 이 1 일 때 RCA 에 올바르게 반영되는지 확인

Input

A = 0000000000000111000000000111000000000000

B = 0000000000001110000011100000000000000000

$$C_{in} = 1$$

Output

S = 0000000000001010100001110011100000000001

Cout = 0

```
+ /tb_rca_40b/tb_a      01000000000000000000111001111000000000000000
+ /tb_rca_40b/tb_b      000000000000000000000000000000001110111111101
    /tb_rca_40b/tb_cn     1
+ /tb_rca_40b/tb_s      01000000000000000011100111101110111111110
    /tb_rca_40b/tb_cout   St0
```

Cin 의 반영 확인_ Cin 이 1 일 때 RCA 에 올바르게 반영되는지 확인하였다.

Testbench 4

1 과 0 이 번갈아 나타나고 C_{in} 이 1 인 경우

Input

A = 10

B = 01

$$C_{in} = 1$$

Output

S = 000

Cout = 1

복잡한 패턴을 테스트하여 RCA 가 이러한 입력에서도 정확히 동작하는지 확인한다.

Testbench 5

만약에 모든 a 의 입력이 1 이고 cin 을 1 을 줬을 때

Input

A = 111

B = 000

$$C_{in} = 1$$

Output

S = 00

Cout = 1

모든 비트가 1 인 경우, 각 비트에서 자리올림이 발생하여 최종적으로 Cout 까지 전달된다.

RCA가 자리올림을 올바르게 전달하는지 확인 할 수 있고,

Cin 이 1 인 경우, 첫 번째 전가산기 에서 발생한 자리올림이 최종적으로 Cout 에 반영되는지 확인할 수 있습니다. 결과적으로 Cin 이 RCA 전체에 걸쳐 영향을 미치는지 확인할 수 있다.